МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ “ЛЬВІВСЬКА ПОЛІТЕХНІКА”

Кафедра систем штучного інтелекту



Лабораторна робота № 3

з дисципліни «Обробка зображень методами штучного інтелекту»

Виконав:

Студент групи КН-415

Лозовий Олег

Викладач:

Пелешко Д.Д.

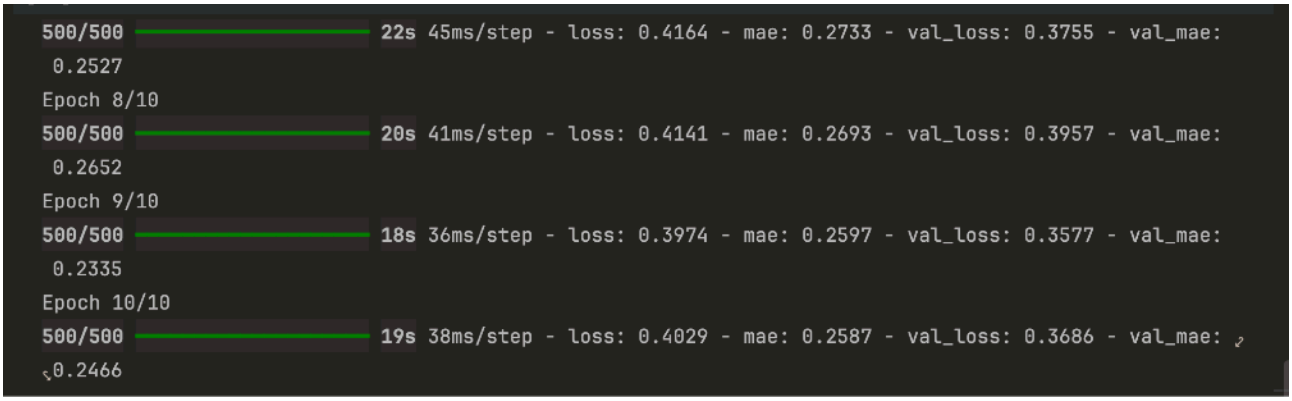
Львів – 2025 р.

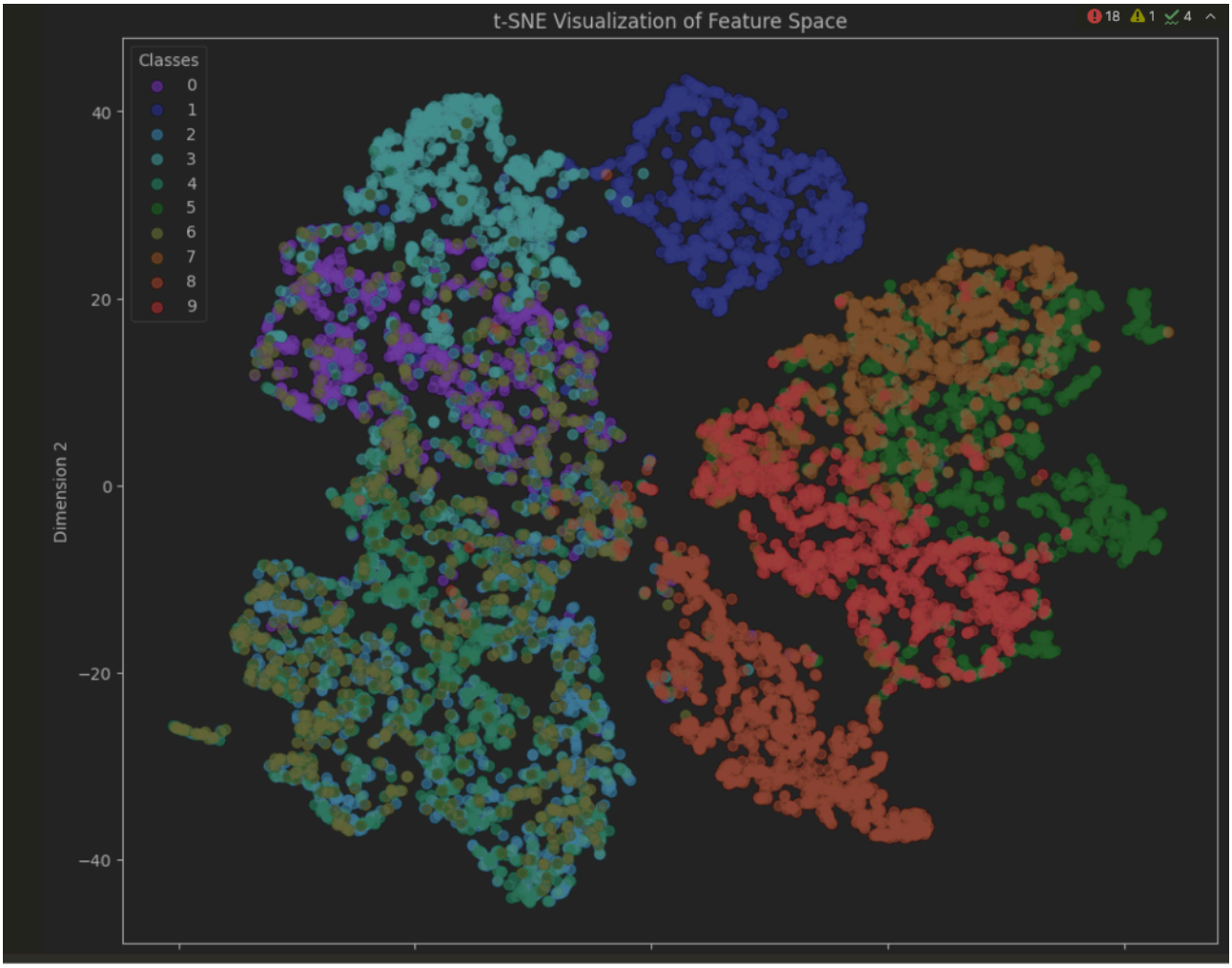
Тема: Класифікація зображень. Застосування нейромереж для пошуку подібних зображень

Мета роботи: набути практичних навиків у розв’язанні задачі пошуку подібних зображень на прикладі організації CNN класифікації

Варіант 12 (1)

Виконати індивідуальне завдання (див. нижче). Зробити налаштування моделі для досягнення необхідної точності. Після цього на базі Siamese networks побудувати систему для пошуку подібних зображень в датасеті fashion-mnist. Візуалізувати отримані результати t-SNE. Номер індивідуального завдання відповідає порядковому номеру студента в журналі групи. Якщо номер перевищує максимальну кількість завдань –береться остача від ділення номера студента на максимальну кількість завдань. Наприклад, якщо порядковий номер студента у списку – 23, а завдань – 15, тоді індивідуальне завдання матиме номер 23 mod 15 = 8.1. Побудувати CNN на основі LeNet-5 для класифікації зображень на основі датасету fashion-mnist.








```
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
```

```
from keras.datasets import fashion_mnist
from keras.models import Sequential, Model
from keras.layers import Conv2D, AveragePooling2D,
Flatten, Dense, Input, concatenate, Batch-
Normalization, Activation, Dropout, MaxPool2D
from keras.utils import to_categorical
from sklearn.manifold import TSNE
(x_train, y_train), (x_test, y_test) =
fashion_mnist.load_data()

x_train = x_train.reshape(-1, 28, 28,
1).astype("float32") / 255.0
x_test = x_test.reshape(-1, 28, 28,
1).astype("float32") / 255.0
y_train_cat = to_categorical(y_train, 10)
y_test_cat = to_categorical(y_test, 10)
# Define LeNet-5 model
lenet_model = Sequential([
    Conv2D(6, kernel_size=5, activation='relu',
input_shape=(28, 28, 1), padding='same'),
    AveragePooling2D(pool_size=(2, 2)),
    Conv2D(16, kernel_size=5, activation='relu'),
    AveragePooling2D(pool_size=(2, 2)),
    Flatten(),
    Dense(120, activation='relu'),
    Dense(84, activation='relu'),
    Dense(10, activation='softmax')
```

```
])
lenet_model.compile(optimizer='adam',
loss='categorical_crossentropy',
metrics=['accuracy'])
lenet_model.fit(x_train, y_train_cat, batch_size=128,
epochs=10, validation_data=(x_test, y_test_cat))
train_groups = [x_train[y_train == i] for i in
np.unique(y_train)]
test_groups = [x_test[y_test == i] for i in
np.unique(y_test)]
def gen_random_batch(in_groups, batch_halfsize=8):
    out_img_a, out_img_b, out_score = [], [], []
    all_groups = list(range(len(in_groups)))
    for match_group in [True, False]:
        group_idx = np.random.choice(all_groups,
size=batch_halfsize)
        if match_group:
            b_group_idx = group_idx
            out_score += [1] * batch_halfsize
        else:
            b_group_idx = [np.random.choice([i for i
in all_groups if i != c_idx]) for c_idx in group_idx]
            out_score += [0] * batch_halfsize
        out_img_a += [in_groups[c_idx]
[np.random.choice(len(in_groups[c_idx]))] for c_idx
in group_idx]
```

```
        out_img_b += [in_groups[c_idx]
[np.random.choice(len(in_groups[c_idx]))] for c_idx
in b_group_idx]
    return np.stack(out_img_a, 0),
np.stack(out_img_b, 0), np.stack(out_score, 0)
```

```
img_in = Input(shape=(28, 28, 1), name='Feature-
Net_ImageInput')
n_layer = img_in
for i in range(2):
    n_layer = Conv2D(8 * 2**i, kernel_size=(3, 3),
activation='linear')(n_layer)
    n_layer = BatchNormalization()(n_layer)
    n_layer = Activation('relu')(n_layer)
    n_layer = Conv2D(16 * 2**i, kernel_size=(3, 3),
activation='linear')(n_layer)
    n_layer = BatchNormalization()(n_layer)
    n_layer = Activation('relu')(n_layer)
    n_layer = MaxPool2D((2, 2))(n_layer)
```

```
n_layer = Flatten()(n_layer)
n_layer = Dense(32, activation='relu')(n_layer)
n_layer = Dropout(0.5)(n_layer)
n_layer = BatchNormalization()(n_layer)
n_layer = Activation('relu')(n_layer)
feature_model = Model(inputs=img_in, outputs=n_layer,
name='FeatureGenerationModel')
```

```
img_a_in = Input(shape=(28, 28, 1), name='Image-
A_Input')
img_b_in = Input(shape=(28, 28, 1), name='Image-
B_Input')
img_a_feat = feature_model(img_a_in)
img_b_feat = feature_model(img_b_in)
```

```
combined_features = concatenate([img_a_feat,
img_b_feat], name='merge_features')
combined_features = Dense(16, activation='relu')
(combined_features)
combined_features = BatchNormalization()
(combined_features)
combined_features = Dense(4, activation='relu')
(combined_features)
combined_features = BatchNormalization()
(combined_features)
combined_features = Dense(1, activation='sigmoid')
(combined_features)
```

```
similarity_model = Model(inputs=[img_a_in, img_b_in],
outputs=combined_features, name='Similarity_Model')
similarity_model.compile(optimizer='adam',
loss='binary_crossentropy', metrics=['mae'])
def siam_gen(in_groups, batch_size=32):
    def generator():
        while True:
```

```
            a, b, sim = gen_random_batch(in_groups,
batch_size // 2)
            yield (a.astype(np.float32),
b.astype(np.float32)), sim.astype(np.float32)
    return tf.data.Dataset.from_generator(generator,
        output_signature=((
            tf.TensorSpec(shape=(None, 28, 28, 1),
dtype=tf.float32),
            tf.TensorSpec(shape=(None, 28, 28, 1),
dtype=tf.float32)
        ), tf.TensorSpec(shape=(None,),
dtype=tf.float32)))
train_ds = siam_gen(train_groups, batch_size=32)
valid_a, valid_b, valid_sim =
gen_random_batch(test_groups, 1024)
loss_history = similarity_model.fit(train_ds,
steps_per_epoch=500, validation_data=([valid_a,
valid_b], valid_sim), epochs=10, verbose=True)
x_test_features = feature_model.predict(x_test,
batch_size=128, verbose=True)
tsne = TSNE(n_components=2, init='pca',
random_state=101, method='barnes_hut', n_iter=500,
verbose=1)
tsne_features =
tsne.fit_transform(x_test_features.reshape(len(x_test
_features), -1))
colors = plt.cm.rainbow(np.linspace(0, 1, 10))
plt.figure(figsize=(12, 10))
for i, color in enumerate(colors):
```

```
    idx = y_test == i
    plt.scatter(tsne_features[idx, 0], tsne_features[idx, 1], color=color, label=str(i), alpha=0.6)
plt.legend(title="Classes")
plt.title("t-SNE Visualization of Feature Space")
plt.xlabel("Dimension 1")
plt.ylabel("Dimension 2")
plt.show()
```